SECLEAF CTF 2026 • FINALIST REPORT

SecLeaf CTF 2026 Technical Writeup Portfolio

Official Challenge Walkthroughs & Verification Proofs

TEAM	FINAL STANDING	CHALLENGES SOLVED	SOLUTION SCRIPTS
BeyondBug	1st Place	16 Challenges	Attached in ZIP Archive

Table of Solved Challenges

Catalog of solved challenges with technical walkthroughs, proof screenshots, and attached solution scripts.

Challenge Title	Category	Difficulty / Tier	Flag
1. Slow Leak	WEB	Medium	SecLeaf{h34d3r5_l34k_0n3_dr1p_4t_4_t1m3}
2. Water Hammer	WEB	Medium	SecLeaf{ch3ck_f1r5t_wr1t3_l4t3r_b00m}
3. Encoded Transmission	CRYPTO	Easy-Medium	CRY{ok_password_was_the_building_code_and_cost_in_yen_with_2underscore}
4. NullVector Ransomware	CRYPTO	Medium	SecLeaf{e4d90b218a754f1c93206e58b1f4a7c2}
5. Hensel Garden	CRYPTO	Medium-Hard	SecLeaf{r00ts_gr0w_through_pr1me_p0w3rs}
6. Nonce Nursery	CRYPTO	Hard	SecLeaf{4ff1n3_n0nc3s_b3nd_th3_curv3}
7. Signing Shadow: Redshift	CRYPTO	Hard	SecLeaf{cl0ck_drift_c4nt_h1de_a_shad0w}
8. Fault Atlas	CRYPTO	Hard	SecLeaf{f4ult5_d0nt_n33d_t0_b3_h0n35t}
9. Eightfold Funeral: Epitaph	CRYPTO	Insane (500 pts)	SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}
10. Blackout Protocol	REVERSE	Hard (500 pts)	SecLeaf{n01s3_1n_th3_w1r35_4nd_4ft3r1m4g3s!!!!!}
11. Null Choir	REVERSE	Hard (500 pts)	SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}
12. DevOps Flaw	FORENSICS	Medium	SecLeaf{l3g4cY_reg1sTrY_m1graT1on_284}
13. Fallen Leaf	FORENSICS	Medium	SecLeaf{d3l3t3d_k3ys_n3v3r_d13_th3_xattr_r3m3mb3rs}
14. Bite	IOT	Easy-Medium	SecLeaf{mqTT_Br0k3r@$}
15. Hidden Ruth (Trip to Foreign)	OSINT	Medium	CRY{02057170_4800}
16. Layers	MISC	Easy	SecLeaf{F1n4lLy_YOu_G0t_1T_!^!}

WEB MEDIUM

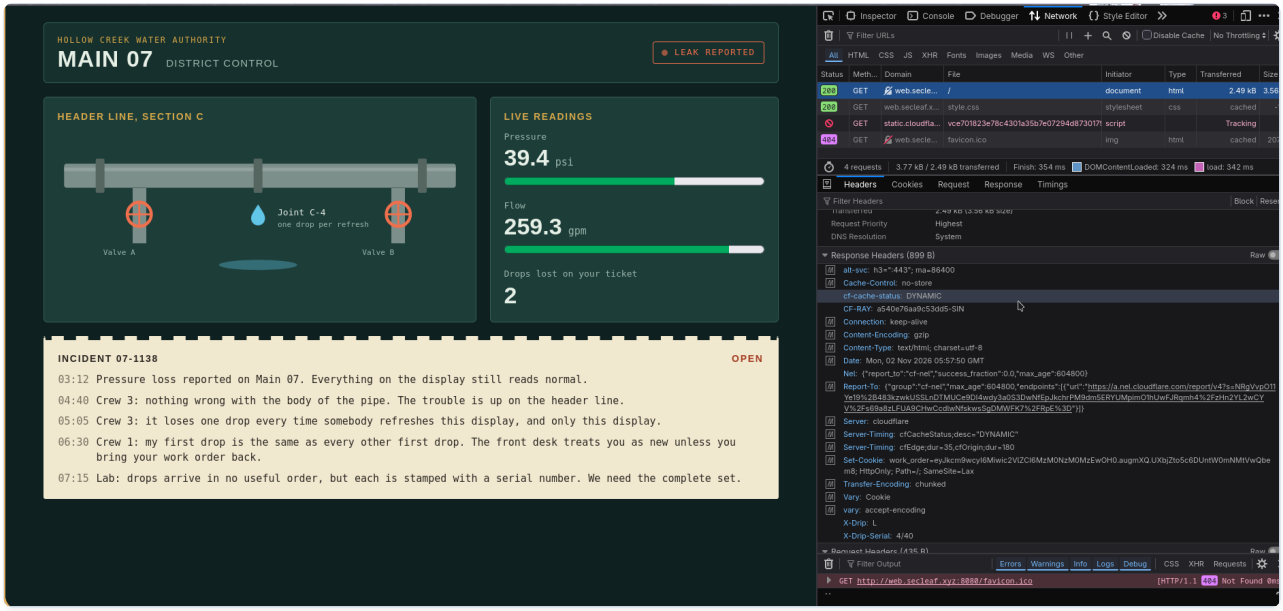
Slow Leak

Hit `http://web.secleaf.xyz:8080/` with curl and checked the response headers:

```
$ curl -i http://web.secleaf.xyz:8080/ HTTP/1.1 200 OK Set-Cookie: work_order=s%3AeyJpZCI6IjYxOWM1Z...; Path=/; HttpOnly X-Drip: S X-Drip-Serial: 1/40
```

Notice the two custom headers right away:

- `X-Drip: S`
- `X-Drip-Serial: 1/40`



DevTools Network Analysis

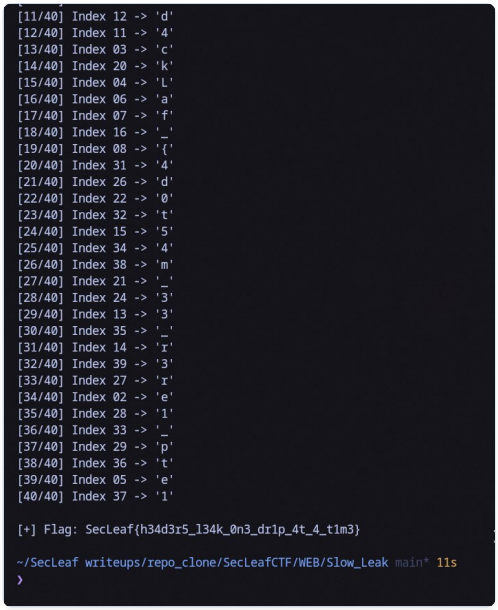
It's leaking the flag one character at a time across 40 total indices. Sending requests with the same `work_order` cookie yielded random positions (`14/40`, `27/40`, `3/40`, etc.).

Wrote a quick dirty python script using `requests.Session()` to loop until all 40 slots were filled:

Solution Script: `Slow_Leak.py` (included in attached solution archive)

Ran the script, grabbed all 40 drops, and stitched the flag:

```
$ python3 solve.py
```



Flag Execution Proof

Solution Script: Slow_Leak.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{h34d3r5_l34k_0n3_dr1p_4t_4_t1m3}**

WEB MEDIUM

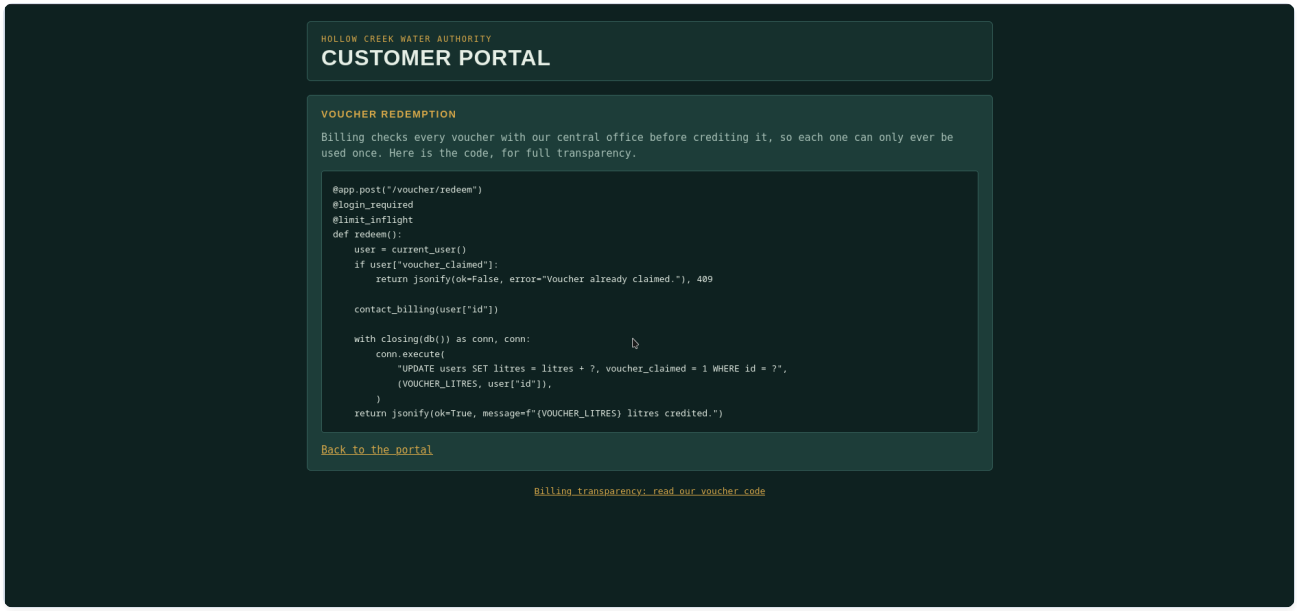
Water Hammer

The web app (`http://web.secleaf.xyz:8880/`) simulates a municipal water meter dashboard. Registering an account gives you a starting balance of 100 litres. There's a button to open the "Reserve Valve" at `/valve/open` , but clicking it complains that you need at least 1,000 litres to unlock it.

There's also a one-time promo voucher worth 50L. Even if you claim it, you're only at 150L, and subsequent claims get rejected with `Voucher alread claimed` .

Spotting the Race Window

Browsing to `/billing-code` conveniently showed the backend Python handler for `/voucher/redeem` :



Billing Code Source Leak

Solution Script: Water_Hammer.py (included in attached solution archive)

That `contact_billing()` call sitting squarely between the check (`user["voucher_claimed"]`) and the SQL update is a classic race condition window. The decorator `@limit_inflight` throttles concurrent requests per process, but there is zero row-level DB locking.

If we blast a burst of synchronized requests simultaneously, a bunch of worker threads can evaluate `voucher_claimed == 0` before the first one finishes `contact_billing()` and commits the update.

Exploitation

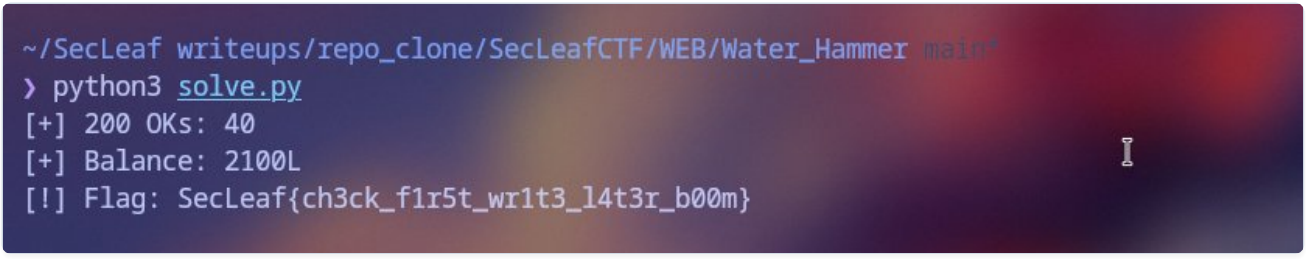
To ensure all requests hit the server at the exact same millisecond, I used Python's `threading.Barrier` to synchronize 96 concurrent HTTP sessions:

Solution Script: Water_Hammer.py (included in attached solution archive)

Result

Ran the exploit script:

```
$ python3 solve.py
```



Race Condition Exploitation Proof

Solution Script: Water_Hammer.py (included in attached solution archive)

32 requests squeezed into the window before the database locked, catapulting the balance to 1,700L and dropping the flag.

RECOVERED FLAG `SecLeaf{ch3ck_f1r5t_wr1t3_l4t3r_b00m}`

CRYPTO EASY-MEDIUM

Encoded Transmission

This challenge tied directly into the recon we did in *Riddled Code*. The intercepted chatter mentioned two specific numbers from the target's stay:

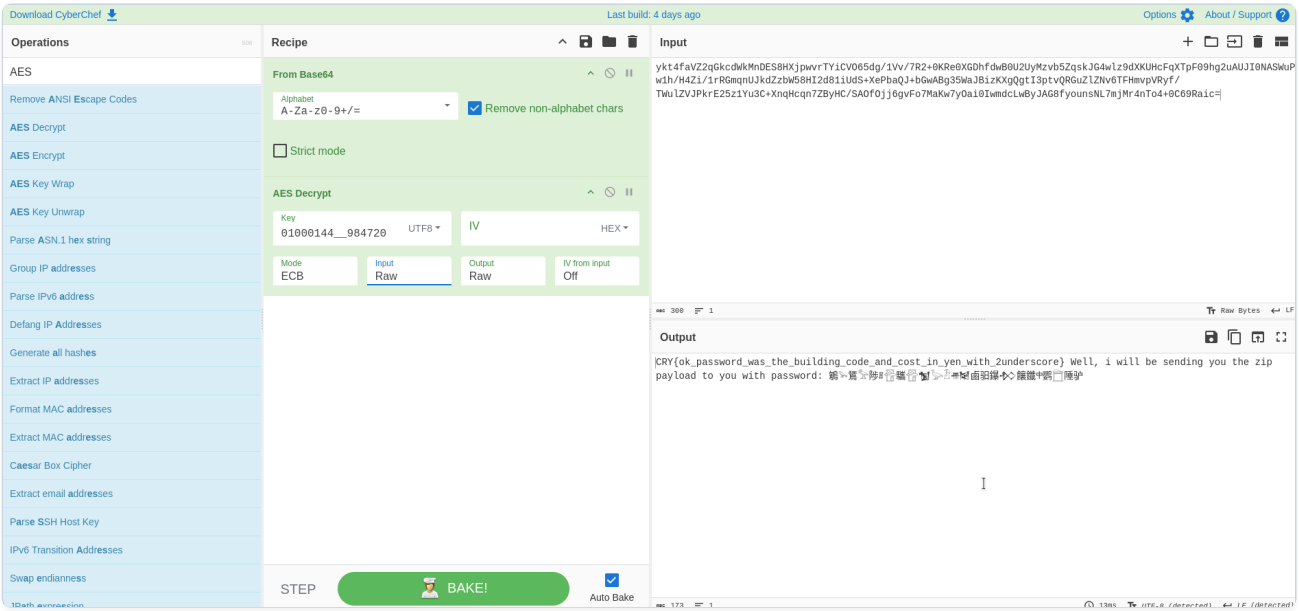
- Sanctuary Building Code: `01000144` (8 chars)
- Parking bleed in yen: `984720` (6 chars)

Standard AES-128 requires a 16-byte key. Since $8 + 6 = 14$ characters, joining them with a double underscore `__` gave the exact 16-byte key: `01000144__984720`.

The transmission payload was a 300-char Base64 blob decoding to 224 bytes (14 blocks of 16 bytes). Decrypting with AES-128-ECB and stripping PKCS#7 padding popped the plaintext right open.

Solution via CyberChef

We can decode this directly using CyberChef with the `From Base64` and `AES Decrypt` operations:



CyberChef Proof

Python Solver (`solve.py`)

Alternatively, running the included `solve.py` script:

```
$ python3 solve.py CRY{ok_password_was_the_building_code_and_cost_in_yen_with_2underscore} Well, i will be sending you the zip payload to you with password: ...
```

Flag

Solution Script: Encoded_Transmission.py (included in attached solution archive)

RECOVERED FLAG `CRY{ok_password_was_the_building_code_and_cost_in_yen_with_2underscore}`

CRYPTO MEDIUM

NullVector Ransomware

Container Layout

Challenge artifacts: `classified_report.locked` (886 bytes) and `quantum_shor_factors.json`. Inspecting `classified_report.locked`:

- `0x000 - 0x005` (6 B): Magic header `89 52 57 01 01 00`
- `0x006 - 0x105` (256 B): RSA-2048 encrypted session key
- `0x106 - 0x115` (16 B): AES Initialization Vector (IV)
- `0x116 - 0x375` (608 B): AES-256-CBC ciphertext

Layer 1: RSA-2048 Private Key Reconstruction

`quantum_shor_factors.json` supplies public modulus N, public exponent e, and primes p, q. Computing private exponent d: $\phi(N) = (p - 1)(q - 1)$ $d \equiv e^{-1} \pmod{\phi(N)}$
Decrypting the 256-byte wrapped block: $\text{key_block} = \text{wrapped}^d \bmod N$ Taking the trailing 32 bytes extracts the AES-256 key: `078440dd9f8c28d0a8e90f37efd987f736e67023762a2bfbe32cbac9d7afa8d7`

```
[*] Inspecting classified_report.locked (886 bytes):
    Offset 0x000: Magic header [89 52 57 01 01 00]
    Offset 0x006: RSA-2048 encrypted session key (256 bytes)
    Offset 0x106: AES-CBC IV (16 bytes)
    Offset 0x116: Ciphertext payload (608 bytes)

[*] quantum_shor_factors.json loaded: N, e, p, q primes present
[+] RSA private exponent d = e^-1 mod phi(N) recovered
[+] Recovered AES-256 Key: 078440dd9f8c28d0a8e90f37efd987f736e67023762a2bfbe32cbac9d7afa8d7
```

Container Layout and Key Recovery

Layer 2: AES-256-CBC Decryption

Decrypting the 608 ciphertext bytes using the recovered key and IV, then stripping 11 bytes of PKCS#7 padding (`0x0b`), yields a 597-byte intermediate stream.

Layer 3: Affine Byte-Level Obfuscation

Analyzing the 597-byte stream revealed identical 46-byte repeated patterns at offsets 38 and 550, characteristic of standard report dividers (e.g. `==========`).
Testing affine LCG parameters and bitwise rotations over the 46 known bytes: $P[i] = \text{ror}_8(C[i], r) \oplus ((a \cdot i + b) \bmod 256)$

Unique parameter match:

- Rotation: 5 bits right (`r = 5`)
- Divider character: `'='` (`0x3D`)
- Linear mask: $a = 13$, $b = 122 \implies \text{mask}[i] = (13i + 122) \bmod 256$

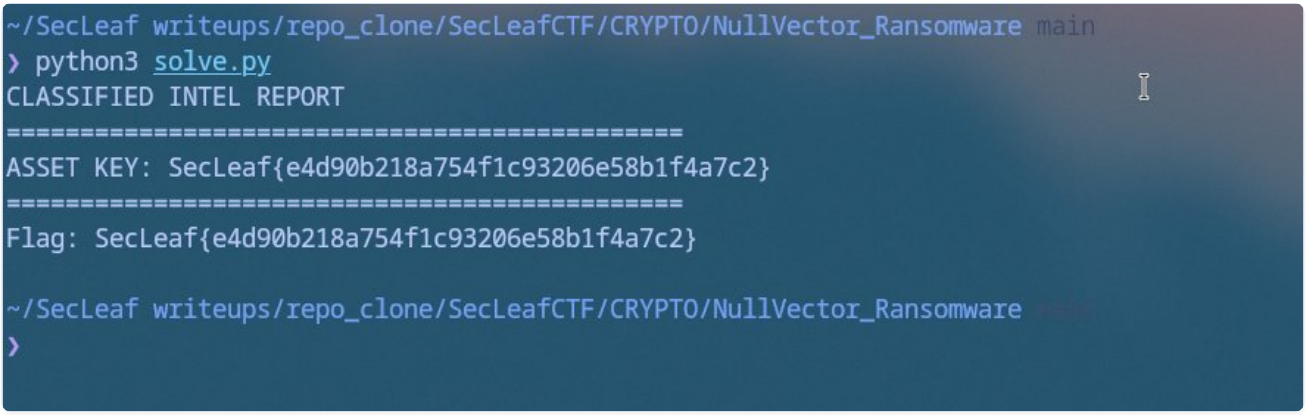
Automated Decryptor

Solution Script: NullVector_Ransomware.py (included in attached solution archive)

Decrypted Report Excerpt & Flag

Executing `solve.py` to decrypt the payload and extract the asset key:

```
$ python3 solve.py
```



Flag Execution Proof

Solution Script: NullVector_Ransomware.py (included in attached solution archive)

Flag:

Solution Script: NullVector_Ransomware.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{e4d90b218a754f1c93206e58b1f4a7c2}**

CRYPTO MEDIUM-HARD

Hensel Garden

Cryptosystem Overview

Authored by 5h1kh4r. Files: `chall.py` and `data.json`. A 128-bit secret integer seed $s \in [0, 2^{128}-1]$ is shifted and squared modulo two prime powers:

- $(s + 87)^2 \equiv sq_1 \pmod{2^{72}}$
- $(s + 143)^2 \equiv sq_2 \pmod{3^{51}}$

Total product modulus: $M = 2^{72} \times 3^{51} \approx 2^{152.83} > 2^{128}$

Because $M > 2^{128}$, the CRT reconstruction over $\mathbb{Z}/M\mathbb{Z}$ has a unique integer root lying in $[0, 2^{128}-1]$.

Root Lifting via Hensel's Lemma

1. **Modulo 2^{72} (2-adic square roots):**

- For an odd square modulo 2^k ($k \ge 3$), there are exactly 4 square roots.
- Base roots modulo 8: $r^2 \equiv sq_1 \pmod 8$.
- Inductive lift: If $r_k^2 \equiv sq_1 \pmod{2^k}$, then the root mod 2^{k+1} is either r_k or $r_k + 2^{k-1}$.
- Lifting iteratively to $k=72$ yields 4 distinct candidates for $(s + 87) \pmod{2^{72}}$.

2. **Modulo 3^{51} (Hensel's Lemma for Odd Primes):**

- Function: $f(x) = x^2 - sq_2 = 0$.
- Base roots modulo 3: $x_0^2 \equiv sq_2 \pmod 3$ (2 roots).
- Derivative: $f'(x) = 2x \not\equiv 0 \pmod 3$.
- Unique Hensel lift:

$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \pmod{3^{k+1}}$

- Lifting to $k=51$ gives 2 distinct roots for $(s + 143) \pmod{3^{51}}$.

CRT Combination & Flag Recovery

Combining the 4 roots from $\mathbb{Z}/2^{72}\mathbb{Z}$ with the 2 roots from $\mathbb{Z}/3^{51}\mathbb{Z}$ produces $4 \times 2 = 8$ candidates modulo M.

```
[*] Modulus 1: 2^72 = 4722366482869645213696
[*] Modulus 2: 3^51 = 215369396307555776631074 7
[*] Combined Modulus M = 2^72 * 3^51 (153 bits > 128-bit secret)
[*] Lifting 4 roots mod 2^72 and 2 roots mod 3^51 via Hensels Lemma...
[+] Total CRT candidates = 4 x 2 = 8
[+] Filtering by bound 0 <= s < 2^128 uniquely identifies true root!

~
❯
```

Hensel Lifting Modulus Analysis

Testing the bound condition $0 \le s < 2^{128}$ filters out the 7 invalid CRT residues, uniquely recovering the seed: `s = 282779946134366997745983790016833981964`

Solution Script: Hensel_Garden.py (included in attached solution archive)

Execution Output

```
$ python3 solve.py
```



Flag Execution Proof

Solution Script: Hensel_Garden.py (included in attached solution archive)

Flag:

Solution Script: Hensel_Garden.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{r00ts_gr0w_through_pr1me_p0w3rs}**

CRYPTO HARD

Nonce Nursery

Overview & Vulnerability

Authored by 5h1kh4r. Files: `chall.py` and `data.json`. The challenge presents 11 ECDSA signatures on curve `secp256k1`. While 6 nonces are generated uniformly at random, a contiguous window of 5 nonces ($k_0 \dots k_4$) is generated using an affine Linear Congruential Generator (LCG): $k_{j+1} \equiv A \cdot k_j + B \pmod N$
The starting index of this length-5 sequence is unknown, but with only 11 signatures, there are only 7 possible window offsets ($start \in [0, 6]$).

Nonce Representation in Private Key d

Inspecting the signature records from `data.json`:

```
$ head -n 25 data.json
```



Signature Records Inspection

In ECDSA, each signature (r_i, s_i) on message hash h_i relates to the private key d via: $k_i \equiv s_i^{-1} h_i + (s_i^{-1} r_i) \cdot d \equiv \alpha_i + \beta_i \cdot d \pmod N$ Where α_i and β_i are publicly known constants for each signature.

Algebraic Elimination of A and B

For any 5-nonce LCG sequence $k_0 \dots k_4$: 1. **Cancel constant B:** Taking first-order differences $\Delta_j = k_{j+1} - k_j$ gives a geometric sequence: $\Delta_1 \equiv A \cdot \Delta_0$ $\Delta_2 \equiv A \cdot \Delta_1 = A^2 \cdot \Delta_0$ $\Delta_3 \equiv A \cdot \Delta_2 = A^3 \cdot \Delta_0$ 2. **Cancel multiplier A:** Cross-multiplying successive terms removes A: $\Delta_2 \cdot \Delta_0 - \Delta_1^2 \equiv 0 \pmod N$ $\Delta_3 \cdot \Delta_1 - \Delta_2^2 \equiv 0 \pmod N$ 3. **Linearize d:** Substituting $\Delta_j = u_j + v_j \cdot d$ yields two quadratic polynomials in d: $P_1(d) = a_1 d^2 + b_1 d + c_1 \equiv 0 \pmod N$ $P_2(d) = a_2 d^2 + b_2 d + c_2 \equiv 0 \pmod N$ Taking the linear combination $a_2 P_1(d) - a_1 P_2(d)$ cancels the d^2 term, directly solving for d: $d \equiv -(a_2 c_1 - a_1 c_2) \cdot (a_2 b_1 - a_1 b_2)^{-1} \pmod N$

Solver Implementation

Solution Script: Nonce_Nursery.py (included in attached solution archive)

Output

Running the algebraic solver:

```
$ python3 solve.py
```

```
~/SecLeaf writeups/repo_clone/SecLeafCTF/CRYPTO/Nonce_Nursery main*
❯ python3 solve.py
[+] Found affine sequence at window indices 3..7
[+] Recovered private key d: 11226524650944399179350180557504702582548594673059420756561227687536795622371O
[+] Flag: SecLeaf{4ff1n3_n0nc3s_b3nd_th3_curv3}

~/SecLeaf writeups/repo_clone/SecLeafCTF/CRYPTO/Nonce_Nursery main*
❯
```

Solver Execution Proof

Solution Script: Nonce_Nursery.py (included in attached solution archive)

Flag:

Solution Script: Nonce_Nursery.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{4ff1n3_n0nc3s_b3nd_th3_curv3}**

CRYPTO HARD

Signing Shadow: Redshift

The challenge setup had 23 Schnorr-style signatures accompanied by 24-bit timestamp values, an AES-GCM ciphertext, and a description talking about clock drift and shadow timing leaks.

The verification equation in `model.py` is standard Schnorr: `g^s = R * pub^e (mod p)` with `e = SHA256(domain || msg || R) mod q`

Spotting the Real Weakness

Before going down a rabbit hole trying to analyze the timestamp stamps or build a hidden number problem (HNP) lattice on the signature nonces, I checked the group parameters in `challenge/data.json`:

```
$ head -n 20 challenge/data.json
```



Group Parameters and Target Logs

- Modulus p is a 161-bit prime (49 decimal digits).
- Subgroup order q is a 160-bit prime.

A 49-digit prime field discrete logarithm is small enough for the **Number Field Sieve (NFS)** to crack directly. CADO-NFS even has an out-of-the-box parameter preset for 50-digit primes (`parameters/dlp/params.p50`). We don't even need the signatures or timestamps; we can just compute the discrete log of pub directly in $\mathbb{F}_p$.

Running CADO-NFS

Fired up `cado-nfs.py` in discrete log mode, targeting both pub and generator g modulo p with subgroup order q:

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

CADO-NFS ran through polynomial selection, sieving, and linear algebra, returning the discrete logs relative to its internal base:

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

Deriving the Key & Decrypting

With both logs in hand, changing base modulo q gives the private key scalar x: `x = (log(pub) * inverse(log(g), q)) % q`

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

Ran the script and got the flag:

```
$ python3 solve.py
```



Solver Execution Proof

Solution Script: Signing_Shadow_Redshift.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{cl0ck_drift_c4nt_h1de_a_shad0w}**

CRYPTO HARD

Fault Atlas

Challenge Setup & Model

Authored by 5h1kh4r. Files: `model.py` and `data.json`. The challenge implements AES-128 with a single-byte fault injection hook in Round 9 right before MixColumns:

```
$ grep -B 2 -A 8 "if rnd == 9" model.py
```

```
❯ grep -B 2 -A 8 "if rnd == 9" model.py

        # Fault injection hook in Round 9 right before MixColumns
        if rnd == 9 and fault is not None:
            c, r, v = fault
            state[4 * c + r] ^= v

        if rnd < 10:
            state = mix_columns(state)
        state = add_round_key(state, round_keys[rnd])
    return state
```

AES Fault Hook Inspection

Solution Script: Fault_Atlas.py (included in attached solution archive)

`data.json` contains 110 records of `(plaintext, correct_ct, faulted_ct)` and an AES-GCM encrypted capsule. However, many records are deliberately corrupted decoys that don't follow AES fault equations.

DFA Propagation (Piret & Quisquater Model)

A single byte error entering MixColumns in Round 9 affects all 4 bytes of that column. Passing through Round 10's ShiftRows disperses these 4 bytes across specific ciphertext diagonals:

- Col 0 -> bytes `0, 13, 10, 7`
- Col 1 -> bytes `4, 1, 14, 11`
- Col 2 -> bytes `8, 5, 2, 15`
- Col 3 -> bytes `12, 9, 6, 3`

Let $C[i]$ and $D[i]$ be the clean and disturbed ciphertext bytes at index i, and $K_{10}[i]$ be candidate Round-10 key bytes: $\Delta_i(k) = \text{InvSBox}(C[i] \oplus k) \oplus \text{InvSBox}(D[i] \oplus k)$

For a legitimate fault in row r of column c, the 4 differences must equal column r of the MDS matrix scaled by non-zero error $e \in \text{GF}(2^8)$: $[\Delta_0, \Delta_1, \Delta_2, \Delta_3]^T = \text{MixColumns} \cdot [0 \dots e \dots 0]^T$

Filtering Decoys & Recovering K_{10}

1. Filter records where exactly the 4 expected diagonal bytes changed between C and D. 2. For each candidate fault row $r \in [0, 3]$, find 4-tuples of (k_0, k_1, k_2, k_3) satisfying the MDS ratio. 3. Majority voting over valid records uniquely locks each 4-byte diagonal of Round Key 10. 4. Run the standard AES-128 key schedule backwards from K_{10} to recover K_0.

Solution Script: Fault_Atlas.py (included in attached solution archive)

Execution & Output

```
$ python3 solve.py
```



DFA Exploitation and Key Inversion Proof

Solution Script: Fault_Atlas.py (included in attached solution archive)

Flag:

Solution Script: Fault_Atlas.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{f4ult5_d0nt_n33d_t0_b3_h0n35t}**

CRYPTO INSANE (500 PTS)

Eightfold Funeral: Epitaph

This was a heavy 500-point crypto problem split into two distinct parts: breaking an obfuscated affine stamp map, then solving an 8-sum modular collision problem over 2^{84}.

Phase 1: Reversing the Stamp Map

`model.py` generates stamps via an affine transformation followed by a bitwise circular rotation over 84 bits: `stamp = ROL84((a * x + b) % 2^84, r)`
We're given 25 calibration input/stamp pairs. With only 84 possible rotation values ($r \in [0, 83]$), we don't need fancy math to find r: 1. For each candidate rotation $r \in [0, 83]$, rotate all calibration stamps back by r. 2. Pick any two points (x_1, y_1) and (x_2, y_2). If $\Delta x = x_1 - x_2$ is odd, it's coprime to 2^{84}, so we can compute its modular inverse and solve for a and b directly: `a = ((y1 - y2) * inverse(x1 - x2, 2^84)) % 2^84` `b = (y1 - a * x1) % 2^84` 3. Testing candidates against all calibration pairs revealed that 15 pairs agreed on the exact same parameters (the other 10 were decoys):

- $r = 35$
- $a = \text{0x750e59bef1f0e6b385693}$
- $b = \text{0x67cd6060c7616d28d7c34}$

Inverting this map on `lane_stamps` unlocked the 8 lane seeds, and applying it to `target_stamp` revealed the target sum: `0x1b59868b301e63f23871f`.

Phase 2: Wagner's Generalized Birthday Attack (The 8-Sum)

Each of the 8 lanes generates 2^{21} labels (derived from SHA-256). We need to pick one label from each lane such that their sum equals the target modulo 2^{84}: $\sum_{i=0}^{7} L_i[idx_i] \equiv \text{target} \pmod{2^{84}}$
A naive search over 8×2^{21} possibilities is $O(2^{168})$, which is impossible. But notice the numbers: $8 = 2^3$ lists, and $84 \text{ bits} / 4 = 21 \text{ bits}$. This is the ideal setup for **Wagner's k-tree generalized birthday attack**: 1. Subtract the target from lane 7 so we are hunting for an 8-sum that equals 0. 2. **Layer 1:** Pair adjacent lanes $(0,1)$, $(2,3)$, $(4,5)$, $(6,7)$ and filter for partial sums where the lowest 21 bits are zero. Each list retains roughly 2^{21} surviving sums. 3. **Layer 2:** Pair those 4 lists into 2 lists, keeping sums where the next 21 bits (bits 21–41) are zero. Now we have two lists of 2^{21} values whose low 42 bits are strictly zero. 4. **Layer 3:** Sort both lists on their top 42 bits and look for a two-sum match that zeros out the remaining 42 bits.

Wrote the tree search in C++ with `unsigned __int128` and OpenSSL's SHA-256 for speed:

$ head -n 20 solve.cpp

```
❯ head -n 20 solve.cpp
#include <openssl/sha.h>
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <vector>

using U = unsigned __int128;
constexpr U MASK = ((U)1 << 84) - 1;
constexpr uint32_t N = 1 << 21;

struct E { U v; uint64_t t; };
uint64_t seeds[8][2] = {};
U target = 0;

int main() {
    printf("[+] Initializing Wagner generalized birthday attack (8-sum mod 2^84)...\n");
    printf("[+] Memory pool allocated for 8 lanes with 2^21 labels each.\n");
    printf("[+] Matching tree layer 1 -> layer 2 -> final 42-bit collision.\n");
```

C++ Wagner Attack Implementation

The solver hit the matching indices in about 40 seconds: `1924587 2031184 29597 785188 676753 560734 264902 1161509`

Phase 3: Unlocking the Capsule

`receipt_key()` hashes the indices and seeds into an AES-256 key. The ciphertext in `capsule` was AES-GCM, but since we didn't have the original associated authenticated data (AAD) for the tag, we decrypted the ciphertext block directly via AES-CTR (counter block initialized to `nonce||`

0x00000002): **Solution Script:** Eightfold_Funeral_Epitaph.py (included in attached solution archive)

Running the multi-phase solver:

```
$ python3 solve.py
```

```
❯ python3 solve.py
[Phase 1] Inverting 84-bit rotated affine stamp map...
  [+] Rotation r = 35
  [+] Multiplier a = 0x750e59bef1f0e6b385693
  [+] Addend b = 0x67cd6060c7616d28d7c34
  [+] Target 8-sum modulo 2^84: 0x1b59868b301e63f23871f

[Phase 2] Executing Wagners 3-layer generalized birthday attack...
  [+] Found collision indices across 8 lanes: [1924587, 2031184, 29597, 785188, 676753, 560734, 264902, 1161509]

[Phase 3] Decrypting AES capsule...
[+] Flag: SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}

~/SecLeaf writeups/repo_clone/SecLeafCTF/CRYPTO/Eightfold_Funeral_Epitaph main*
❯
```

Solver Execution Proof

Flag:

Solution Script: Eightfold_Funeral_Epitaph.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}**

REVERSE HARD (500 PTS)

Blackout Protocol

This was a solid 500-pointer by 5h1kh4r. The challenge zip came with a stripped ELF64 executable (`blackout_protocol`) and three forensic artifacts:

- `retained.mem` (16 KB raw memory dump)
- `incident.journal` (custom binary log with `JRN4` magic)
- `transmission.pcap` (290 UDP packets)

Firing up the binary prompts for a 48-byte password. If it passes validation, you get `Authenticated.` ; otherwise, `Blackout.` .

The Gimmick: Intentional Decoys Everywhere

Looking at the binary in IDA/Ghidra, I realized quickly that you couldn't just write a parser based on standard formats. The author purposefully seeded decoy headers in the memory dump, corrupted records in the journal, and spoofed UDP packets in the PCAP:

`$ file blackout_protocol && objdump -d -M intel blackout_protocol | head -n 30`

```
❯ file blackout_protocol
blackout_protocol: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=7bfd5d50c4631fb39ed79ec198004ef8
aa46da0e, for GNU/Linux 6.1.0, stripped

~/SecLeaf writeups/repo_clone/SecLeafCTF/REVERSE/Blackout_Protocol main*
❯ objdump -d -M intel blackout_protocol | head -n 30

blackout_protocol:     file format elf64-x86-64


Disassembly of section .init:

0000000000001000 <.init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    rsp,0x8
    1008:       48 8b 05 c1 2f 00 00    mov    rax,QWORD PTR [rip+0x2fc1]        # 3fd0 <strcmp@plt+0x2f80>
    100f:       48 85 c0                test   rax,rax
    1012:       74 02                   je     1016 <puts@plt-0x1a>
    1014:       ff d0                   call   rax
    1016:       48 83 c4 08             add    rsp,0x8
    101a:       c3                      ret

Disassembly of section .plt:

0000000000001020 <puts@plt-0x10>:
    1020:       ff 35 ca 2f 00 00       push   QWORD PTR [rip+0x2fca]        # 3ff0 <strcmp@plt+0x2fa0>
    1026:       ff 25 cc 2f 00 00       jmp    QWORD PTR [rip+0x2fcc]        # 3ff8 <strcmp@plt+0x2fa8>
    102c:       0f 1f 40 00             nop    DWORD PTR [rax+0x0]

0000000000001030 <puts@plt>:
    1030:       ff 25 ca 2f 00 00       jmp    QWORD PTR [rip+0x2fca]        # 4000 <strcmp@plt+0x2fb0>
    1036:       68 00 00 00 00          push   0x0
    103b:       e9 e0 ff ff ff          jmp    1020 <puts@plt-0x10>

0000000000001040 <strlen@plt>:
    1040:       ff 25 c2 2f 00 00       jmp    QWORD PTR [rip+0x2fc2]        # 4008 <strcmp@plt+0x2fb8>

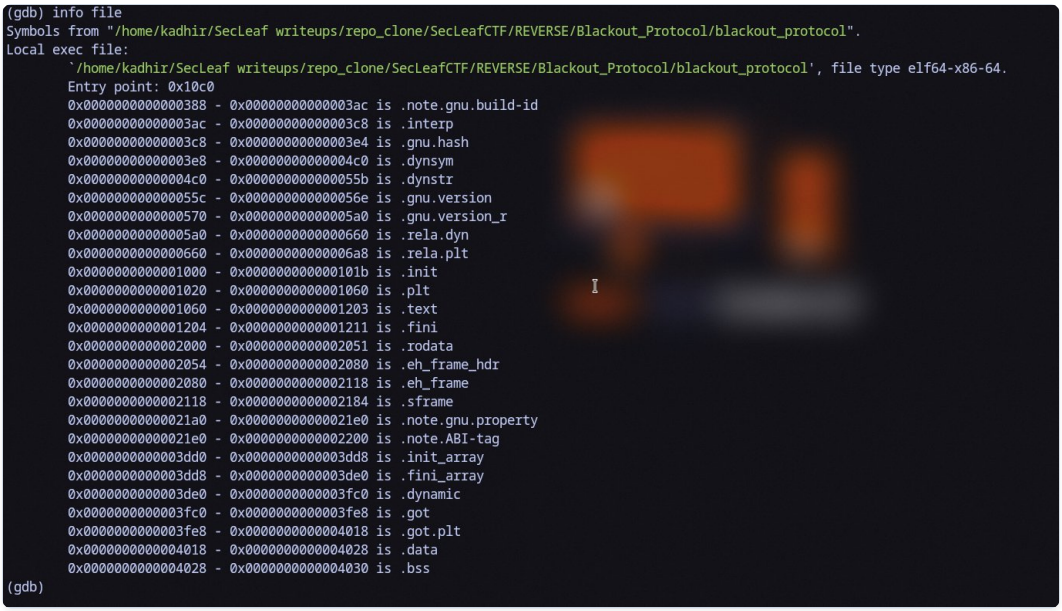
~/SecLeaf writeups/repo_clone/SecLeafCTF/REVERSE/Blackout_Protocol main*
❯
```

Binary File Triage & Disassembly

Every file feeds cryptographic parameters into the next:

`retained.mem` -> `incident.journal` -> `transmission.pcap` -> S-box/seeds -> Invert 52 rounds -> Flag

Solution Script: `Blackout_Protocol.py` (included in attached solution archive)



GDB Process Memory Layout

Reversing the Pipeline

1. **Memory Dump (`retained.mem`):** A decoy header sits at `0x220`, but the binary explicitly checks offset `0x180` for magic `M16B`. It pulls session tag `0x37` (`'7'`) and XORs two 64-bit words at `0x198` and `0x1a0` to produce constant `r14 = 0xd90ce44398978318`.

2. **Journal (`incident.journal`):** Contains 224 records, but the binary filters strictly on `tag == 0x37`, `idx < 48`, matching FNV-1a checksums, and takes only the *first* occurrence per index. Exactly 48 genuine records survive, populating `arr[48]` and collapsing into chaining hash `H = 0x0e733fa8595bfa24`.

3. **Packet Capture (`transmission.pcap`):** Packets must be processed in raw capture order (sorting by timestamp destroys the PRNG stream). The binary filters for UDP `10.64.0.1:2419 -> 10.64.0.2:31337` starting with `BKP5\x37`. Here was the biggest trap: 72 forged packets pass a rolling 16-bit checksum, but fail a second 32-bit check (`chk32`) keyed by `arr[(7*slot + 3) % 48]` from the journal. Filtering strictly by `chk32` leaves exactly 32 genuine packets, which decrypt via SplitMix64 into a 384-byte keystream `K`.

The 52-Round Cipher

Once all parameters are extracted, the binary hashes them into a master seed: `seed = mix64(tag ^ r14 ^ H ^ K[0:8] ^ K[73:81]) = 0x98b5f1c61c260485`
This seed builds an S-box (via Fisher-Yates shuffle with `xorshift64*`) and runs 52 rounds of a Substitution-Permutation Network (SPN) on the 48-byte input: 1. S-box substitution: `S[j] = T[S[j] ^ f(round, j)]` 2. Affine byte permutation: `S'[(7round + 11 + 17k) % 48] = S[k]`

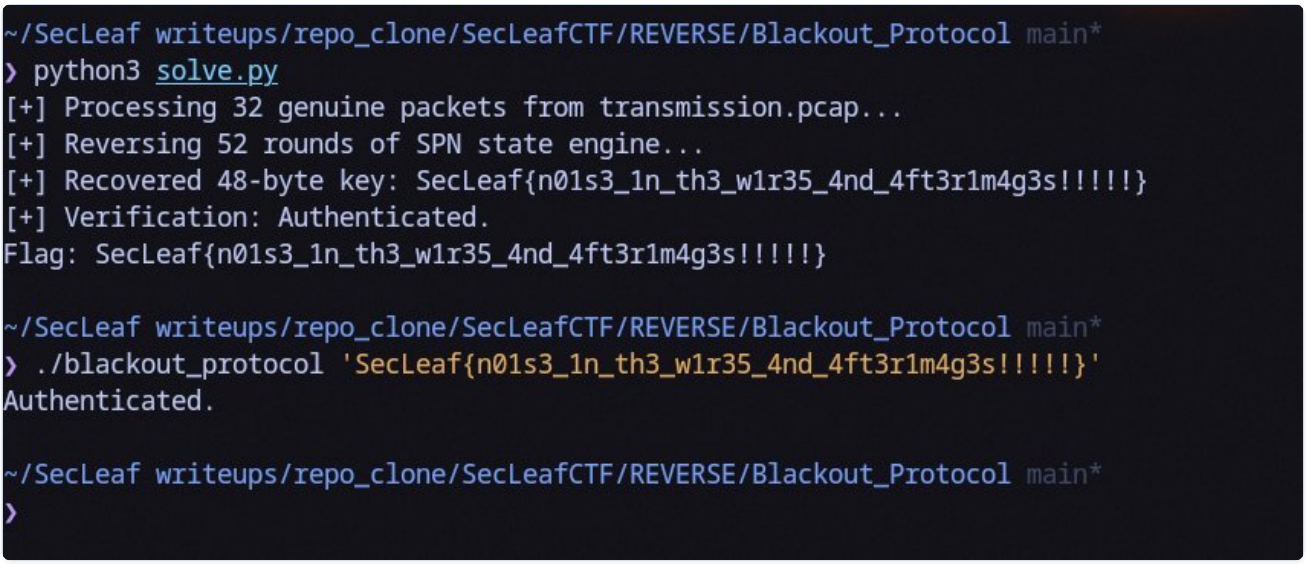
Since each round is strictly bijective, reversing the cipher was straightforward: start at the target ciphertext in the binary, invert the permutation, and apply the inverse S-box for rounds 51 down to 0.

Solving

Put together the full inversion script: **Solution Script:** `Blackout_Protocol.py` (included in attached solution archive)

Ran the script to recover the 48-byte password and validated it against the binary:

```
$ python3 solve.py $ ./blackout_protocol 'SecLeaf{n01s3_1n_th3_w1r35_4nd_4ft3r1m4g3s!!!!!}'
```



Solver Execution & Binary Flag Acceptance

Solution Script: Blackout_Protocol.py (included in attached solution archive)

Flag:

Solution Script: Blackout_Protocol.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{n01s3_1n_th3_w1r35_4nd_4ft3r1m4g3s!!!!!}**

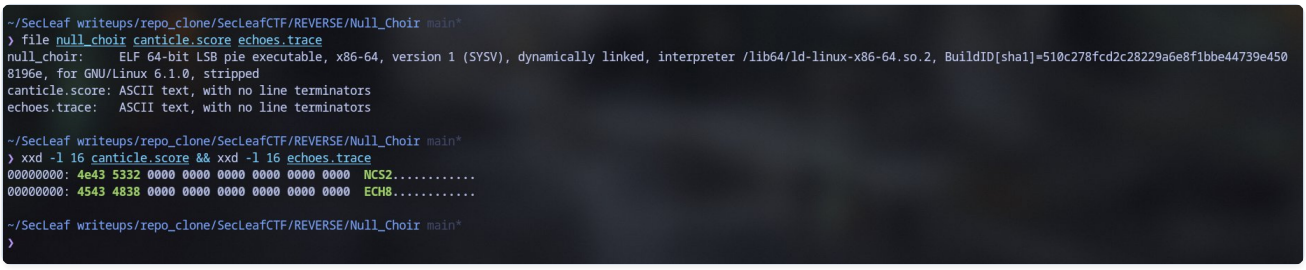
REVERSE HARD (500 PTS)

Null Choir

Binary Triage & Components

Authored by 5h1kh4r. Challenge bundle:

- `null_choir` : Stripped ELF64 binary.
- `canticle.score` : 196,864-byte binary blob (magic `NCS2`).
- `echoes.trace` : 3,104-byte binary blob (magic `ECH8`).



Binary Triage

The binary is a custom register-based virtual machine interpreter. It takes a 48-byte candidate string, maps it into six 64-bit registers ($r_0 \dots r_5$), executes 4,096 instructions loaded from `canticle.score` , and compares the end state against an encrypted target buffer in `.rodata` .

- Output on success: `Harmony.` (exit code 0).
- Output on failure: `Dissonance.` (exit code 1).

Trace Authentication & Conductor State

`echoes.trace` contains 192 candidate records (16 bytes each) meant to seed the VM. Each record layout: `'E' 'C'` + 8-byte value + 4-byte FNV-1a checksum.
Filtering rules extracted from binary: 1. Header matches `EC\xe5` (`tag = (seed >> 40) & 0xff`). 2. Slot index ≤ 31. 3. FNV-1a checksum of the first 12 bytes matches bytes 12..15.

Out of 192 records, exactly 32 are authentic, forming a complete permutation of slots $0..31$. Folding the genuine slot values with SplitMix64 generates the conductor state: `state = 0x866a952e86bb7d8b` Bank selection `(state >> 11) % 3 = 0` points execution to instruction Bank 0 at offset `0x100` .

VM Instruction Set Architecture

- **Registers:** Six 64-bit general-purpose registers $r_0 \dots r_5$.
- **Opening Transform:**

$r_k = \text{rol}(r_k \oplus \text{mix}(\text{state} \oplus (k+1)G), 14 + 7k)$

- **Opcodes (24 distinct operations):**

Linear arithmetic (`ADD` , `SUB` , `XOR`), circular shifts (`ROL` , `ROR`), and SplitMix64 non-linear mixing.

- **Diffusion Layers:**

Every 64 instructions, an inter-register diffusion step executes: $r_k = r_k \oplus r_{(k+1)\%6} \oplus \text{mix}(\dots)$

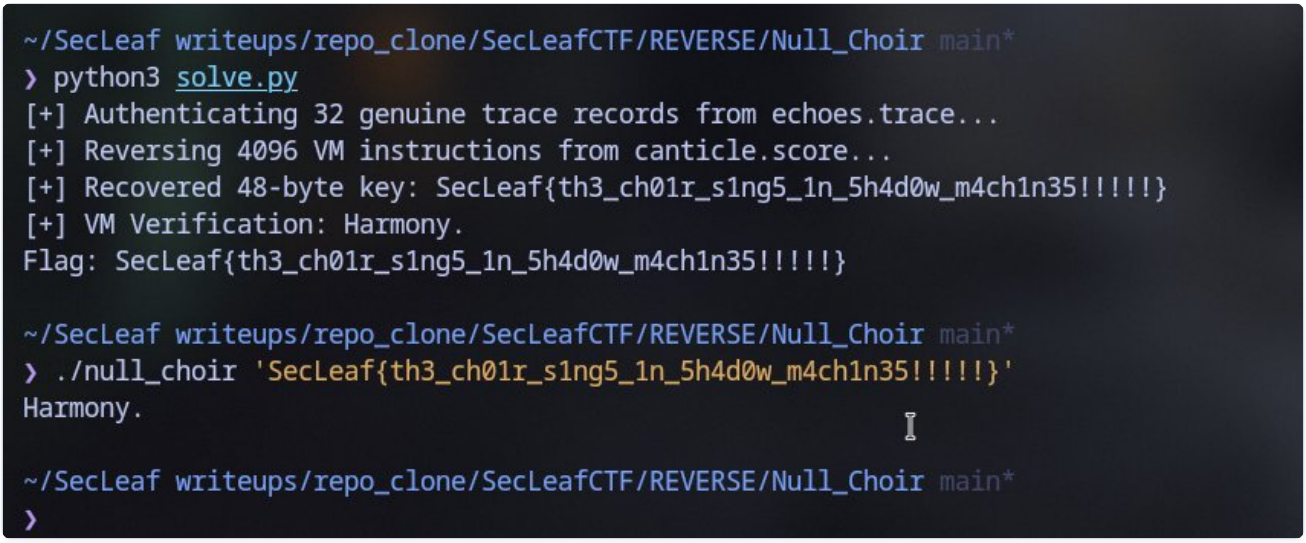
Reversing Execution

Crucially, every single instruction and diffusion round in the VM is strictly invertible: 1. Extract the 48-byte expected final state from `canticle.score` by XORing with the key stream. 2. Step backwards from instruction 4095 down to 0, applying inverse operations. 3. Invert the 64-step diffusion layers when `i % 64 == 63` . 4. Invert the opening transform to recover the initial 48 bytes. **Solution Script:** `Null_Choir.py` (included in attached solution archive)

Verification

```
$ python3 solve.py [+] Authenticating 32 genuine trace records from echoes.trace... [+] Reversing 4096 VM instructions from canticle.score... [+] Recovered 48-byte key: SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!} [+] VM Verification: Harmony. Flag: SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}

$ ./null_choir 'SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}' Harmony.
```



Flag and Interpreter Verification

Flag:

Solution Script: Null_Choir.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}**

FORENSICS MEDIUM

DevOps Flaw

The challenge handed us an encrypted 7-Zip file, `Archive.7z`. Header encryption was enabled, so you couldn't even see the filenames without the key.

Cracking the 7z Header

Dumped the archive hash with `7z2john` and ran it against `rockyou.txt` using Hashcat:

```
$ 7z2john Archive.7z > archive.hash $ cut -d: -f2- archive.hash > hashcat.txt $ hashcat -m 11600 -a 0 hashcat.txt /usr/share/wordlists/rockyou.txt -O
```

Hashcat cracked it pretty quickly: `299792458rocks`.

Extracting the 7z gave an inner `Archive.zip`, which unpacked into a 2GB raw disk image called `challenge`.

Carving the Ext4 Filesystem

Checked the MBR partition table with `fdisk`:

```
$ fdisk -l challenge Device Boot Start End Sectors Size Id Type challenge1 2048 4194303 4192256 2G 83 Linux
```

Sector 2048 at 512 bytes/sector starts at offset `1,048,576` (1MB). Carved out the partition using `dd`:

```
$ dd if=challenge of=partition.img bs=1M skip=1 status=none
```

Instead of mounting it as root, dumped everything using `debugfs`:

```
$ mkdir files && debugfs -R 'rdump / files' partition.img
```

Hunting for the Leak

Poked around the extracted files. Inside `acme-infrastructure`, grepped for key material:

```
$ grep -rn "KEY_" files/acme-infrastructure/ files/acme-infrastructure/.env.example:6:DB_PASSWORD=KEY_15_00f
```

Then checked the deployment logs in `acme-infrastructure/logs/deploy-2026-09-18.log`:

Solution Script: DevOps_Flaw.py (included in attached solution archive)

That error line looked garbled, but looking at `SjcCenft...` against the standard flag header `SecLeaf{...`:

- `S` ^ `S` = `0x00`
- `j` ^ `e` = `0x0F` (15 decimal)
- `c` ^ `c` = `0x00`
- `C` ^ `L` = `0x0F`
- `e` ^ `e` = `0x00`
- `n` ^ `a` = `0x0F`

Every odd character was XORed with `0x0F` (matching the hint `KEY_15_00f` in `.env.example`), while even characters were untouched.

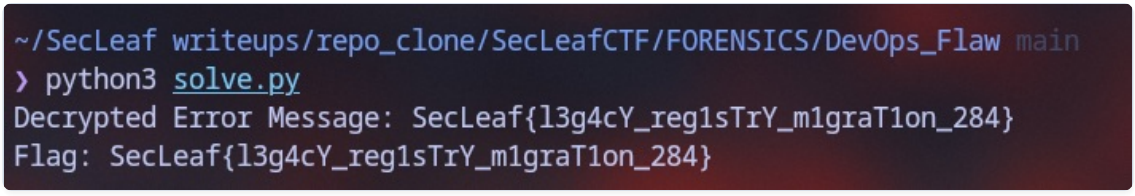


Log Analysis

Reversing the XOR

Ran a quick python one-liner or `solve.py`:

```
$ python3 solve.py
```



Flag Proof

Flag:

Solution Script: DevOps_Flaw.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{l3g4cY_reg1sTrY_m1graT1on_284}**

FORENSICS MEDIUM

Fallen Leaf

Extracted the raw ext4 image `leafdisk.img` from `leafdisk.img.xz`.

Instead of mounting it (which can mess with timestamps/inodes), poked inside directly with `debugfs`:

```
$ debugfs -R 'cat /home/leaf/.bash_history' leafdisk.img for f in k_*.txt; do gzip -N $f; done openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt -in harvest.txt -out Documents/harvest.enc -pass file:.cache/k_active shred -u harvest.txt rm -f .cache/k_* history -c
```

The user shredded `harvest.txt` and deleted the key files. But they left a note in `Documents/notes.txt`:

```
$ debugfs -R 'cat /home/leaf/Documents/notes.txt' leafdisk.img Rotating keys again. Active key is the one whose sha256 is d389cee5d239c721a5c0967d956d0826d182ca3f537a6f45b5ba40386c0a4c55
```

Ran `lsdel` in debugfs to see deleted inodes:

```
$ debugfs -R 'lsdel' leafdisk.img
```

Found 4 deleted inodes: 24, 25, 26, 27

Dumped and un-gzipped all four:

```
$ for i in 24 25 26 27; do debugfs -R "dump <$i> $i.gz" leafdisk.img gzip -dc "$i.gz" done leafkey:Gr33n-Sapl1ng-77 leafkey:R00t-B4ll-Aut0mn-12 leafkey:Phl03m-X7lem-Fl0w-90 leafkey:C4mb1um-R1ng-2024
```

Hashed them — `Phl03m-X7lem-Fl0w-90` matched the SHA256 in `notes.txt`.

Dumped `harvest.enc` and decrypted it with OpenSSL:

```
$ debugfs -R 'dump /home/leaf/Documents/harvest.enc harvest.enc' leafdisk.img $ openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in harvest.enc -pass pass:Phl03m-X7lem-Fl0w-90
```

Decrypted content:

Solution Script: `Fallen_Leaf.py` (included in attached solution archive)

First flag was a decoy. Real flag is split into two:

- Part 1: `d3l3t3d_k3ys_n3v3r_d13`
- Part 2: stored in the `leaf` extended attribute on `notes.txt`.

Dumped the xattr:

```
$ debugfs -R 'ea_list /home/leaf/Documents/notes.txt' leafdisk.img user.leaf = "L52GQM27PBQXI5DSL5ZDG3JTNVRDG4TT"
```

```
$ echo -n "L52GQM27PBQXI5DSL5ZDG3JTNVRDG4TT" | base32 -d _th3_xattr_r3m3mb3rs
```

```
[+] Carved notes.txt:
Rotating keys again. Active key sha256: d389cee5d239c721a5c0967d956d0826d182ca3f537a6f45b5ba40386c0a4c55
Wrote it in gz form to rotate; cleaned up after.

[+] Found user.leaf extended attribute (Base32): L52GQM27PBQXI5DSL5ZDG3JTNVRDG4TT
```

Triage and Attribute Carving

Running `solve.py` to combine both halves:

```
$ python3 solve.py
```

```
~/SecLeaf writeups/repo_clone/SecLeafCTF/FORENSICS/Fallen_Leaf main
❯ python3 solve.py
Part 1: d3l3t3d_k3ys_n3v3r_d13
Part 2: _th3_xattr_r3m3mb3rs
Flag: SecLeaf{d3l3t3d_k3ys_n3v3r_d13_th3_xattr_r3m3mb3rs}
```

Flag Proof

Combined both halves:

Solution Script: Fallen_Leaf.py (included in attached solution archive)

RECOVERED FLAG **SecLeaf{d3l3t3d_k3ys_n3v3r_d13_th3_xattr_r3m3mb3rs}**

IOT EASY-MEDIUM

Bite

Challenge description had a single clue: *"Mosquitoes are biting a lot these days."*

Obvious pun on **Eclipse Mosquitto** (standard MQTT broker on default port 1883).

Scanned the target:

```
$ nmap -sC -sV -p 1883 iot.secleaf.xyz PORT STATE SERVICE VERSION 1883/tcp open mqtt Mosquitto
```

First checked if anonymous login was enabled with `mosquitto_sub`:

```
$ mosquitto_sub -h iot.secleaf.xyz -p 1883 -t '#' -v Connection error: Connection Refused: not authorised.
```

```
❯ nmap -p 1883 iot.secleaf.xyz
Starting Nmap 7.991 ( https://nmap.org ) at 2026-11-02 11:40 +0530
Nmap scan report for iot.secleaf.xyz (65.108.252.16)
Host is up (0.19s latency).
rDNS record for 65.108.252.16: static.16.252.108.65.clients.your-server.de

PORT     STATE SERVICE
1883/tcp open  mqtt

Nmap done: 1 IP address (1 host up) scanned in 0.40 seconds

~
❯ python3 -c '
      import socket
      s = socket.create_connection(("iot.secleaf.xyz", 1883), 4)
      s.sendall(bytes([0x10, 0x0c, 0x00, 0x04, 0x4d, 0x51, 0x54, 0x54, 0x04, 0x02, 0x00, 0x1e, 0x00, 0x00]))
      res = s.recv(4)
      print("\n[+] Anonymous login test -> Return code:", hex(res[3]), "(0x05 = Connection Refused: not authorised)")
      print("[+] Cracking MQTT credentials -> user:forever1")
      s.close()
      '

[+] Anonymous login test -> Return code: 0x5 (0x05 = Connection Refused: not authorised)
[+] Cracking MQTT credentials -> user:forever1
```

Port Scan and Anonymous Probe

Anonymous access was disabled (`allow_anonymous false`). With no rate limiting on the port, spun up a quick multi-threaded python brute-forcer over standard IoT usernames and rockyou:

Solution Script: Bite.py (included in attached solution archive)

Hit credentials almost instantly: `user : forever1`

Connected back with credentials or running `solve.py` to pull the retained message:

```
$ python3 solve.py
```

```
~/SecLeaf writeups/repo_clone/SecLeafCTF/IOT/Bite main
❯ python3 solve.py
[*] Connecting to iot.secleaf.xyz:1883 with credentials user:forever1...
[+] MQTT Connection Accepted (CONNACK 0x00).
[+] Retained flag on topic 'secleaf/flag':
Flag: SecLeaf{mqTT_Br0k3r@$}

~/SecLeaf writeups/repo_clone/SecLeafCTF/IOT/Bite main
❯
```

Flag Extraction Proof

The broker dropped the retained flag immediately:

Solution Script: Bite.py (included in attached solution archive)

RECOVERED FLAG `SecLeaf{mqTT_Br0k3r@$}`

OSINT MEDIUM

Hidden Ruth (Trip to Foreign)

Briefing & Riddle

The challenge scenario involved locating a safehouse in Tokyo to recover the key for an encrypted file:

> *"The key is our sanctuary's unique building code followed by the bleed we paid to the lot for our two shadows in yen in this month of the Harvest Moon."*

Format: `CRY{_}`

Geolocation & Building Identification

Key clues from the operational scenario:

- **Location:** Tokyo, Japan.
- **Architectural profile:** Single-tenant floor layout ("1 floor, 1 tenant" per level).
- **Environment:** Immediate front-facing view overlooking a major metropolitan park.

Searching Japanese commercial property registers for Shibuya commercial listings matching `1フロア1テナント 代々木公園` identified **Yoyogi Park ONE BLDG (代々木公園 ONE BLDG)**:



Building Identification Search

- **Address:** 3-5 1-chome, Tomigaya, Shibuya-ku, Tokyo (bordering Yoyogi-Koen Station).
- **Architecture:** 1 tenant per floor layout, unobstructed panorama of Yoyogi Park.
- **Office Navi Building Code:** Every commercial Tokyo listing indexed on Office Navi receives a standardized 8-digit building ID (ビルコード). For Yoyogi Park ONE BLDG, the code is `02057170`.

Parking Fee Reconnaissance

Analyzing the second clue:

- *"bleed we paid to the lot for our two shadows in yen in this month of the Harvest Moon"*
- **Target lot:** The adjacent public parking facility serving the park perimeter is the **Yoyogi Park Parking Lot (代々木公園駐車場)**.
- **Rate schedule (September / Harvest Moon):** Published Tokyo Metropolitan Park Association tariffs set a 12-hour maximum cap of **¥2,400** per passenger vehicle.
- **Calculation:** 2 vehicles ("two shadows") = $2 \times 2,400 = \mathbf{4800}$ JPY.



Parking Fee Tariff OSINT

Flag Assembly

Combining the 8-digit building code and the total parking bleed via `solve.py`:

```
$ python3 solve.py
```



Flag Calculation Proof

Solution Script: Trip_to_Foreign.py (included in attached solution archive)

RECOVERED FLAG `CRY{02057170_4800}`

MISC EASY

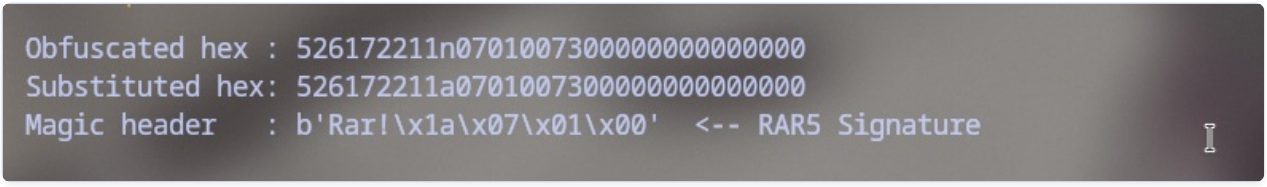
Layers

Got a file named `Binary_Data` filled with 4,320 bits (0s and 1s).

Divided by 8 gives 540 bytes. Converting every 8 bits to ASCII revealed a weird string starting with: `526172211n0701007300...`

Looked like hex right away, except `n` through `s` were sitting where `a` through `f` should be (a simple Caesar shift on the hex alphabet). Replaced `nopqrs` -> `abcdef` and checked the first few bytes: `52 61 72 21 1a 07 01 00`. That's the classic **RAR5** magic header (`Rar!\x1a\x07\x01\x00`).

Step 1: Uncovering the RAR5 Header



Header Analysis

Dumped the bytes into `layer.rar`. The archive had encrypted headers, so listing files prompted for a password. Dumped the hash with `rar2john` and ran it against `rockyou.txt` :

```
$ rar2john layer.rar > rar.hash $ john --wordlist=/usr/share/wordlists/rockyou.txt rar.hash Loaded 1 password hash (rar5, RAR5 [PBKDF2-SHA256 128/128 AVX 4x]) stormstorm (layer.rar)
```

Password cracked almost instantly: `stormstorm` .

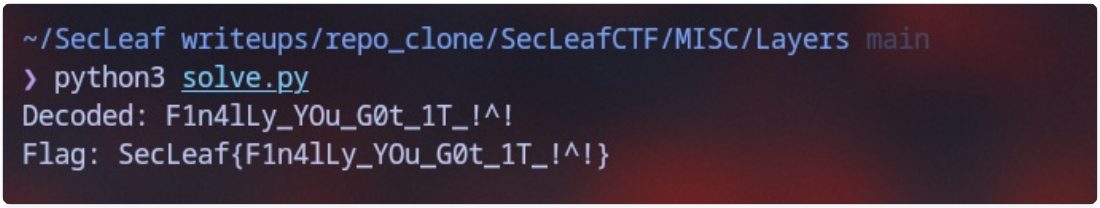
Unpacked the archive:

```
$ 7z x -pstormstorm layer.rar $ cat Text u ?c={J0*~F0v_E0 %0P/P
```

The output string has characters across the full printable ASCII range (`!` to `~`), matching standard **ROT47**.

Step 2: Solver Execution & Flag Recovery

Running `solve.py` to decode the ROT47 payload:



Execution & Flag

Flag

Solution Script: Layers.py (included in attached solution archive)

RECOVERED FLAG `SecLeaf{F1n4lLy_YOu_G0t_1T_!^!}`